

P3715R0

Tightening floating-point semantics for C++

Draft Proposal, 2025-04-18

This version:

<http://wg21.link/p3715r0>

Author:

[Joshua Cranmer](#)

Audience:

SG6, SG22, EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Floating-point semantics are hard, and C++ is almost entirely silent on the matter at present. This is a paper to start the process of fixing that silence, by beginning with a comprehensive overview of the current situation in order to take a holistic approach instead of a piecemeal approach for floating-point semantics.

Table of Contents

1	Revision history
2	Introduction
3	Background
3.1	IEEE 754
3.1.1	Non-IEEE 754 types
3.2	Hardware implementation
3.3	Compiler implementation
3.4	Programming language semantics
3.5	IEEE 754 rules for language specifications
4	Comparison of language standards
5	Motivation
5.1	Implementers' perspective
5.1.1	Excess precision
5.1.2	Denormal flushing
5.1.3	Associativity and vectorization
5.1.4	FMA contraction
5.1.5	Fast-math
5.1.6	Constant expressions
5.1.7	Type traits
5.2	Users' perspective
5.2.1	Reproducible results
5.2.2	Rounding mode
5.2.3	Environment access

6 Solution space

- 6.1 Do nothing
- 6.2 Unspecified behavior
- 6.3 Demand strict conformance
- 6.4 Pragmas
- 6.5 Attributes
- 6.6 Fundamental types
- 6.7 Type wrappers
- 6.8 Free functions
- 6.9 Special lambdas

7 Proposal

- 7.1 Floating-point formats
- 7.2 Excess precision
- 7.3 Denormal flushing
- 7.4 Fast-math
- 7.5 Associativity
- 7.6 FMA contraction
- 7.7 Constant expression rules
- 7.8 Type traits
- 7.9 Rounding mode
- 7.10 Reproducible results
- 7.11 Environment access

8 Questions

§ 1. Revision history

R0: first version

§ 2. Introduction

The C++ standard, at present, says extremely little about floating-point semantics. In recent meetings, however, there has been a few tracks of papers that are trying to clarify the behavior of floating-point, starting with P2746 on proposing to abandon the current C functions for using the rounding mode and replacing it with a new facility for handling rounding. This has been followed with P3375, P3479, P3488, and P3565 on various aspects of floating-point.

In the course of discussing these papers, the committee has signalled an intent to firm up the specification of floating-point semantics. However, many of the issues of floating-point are somewhat related to one another, and without an understanding of all of these issues, the risk is that the committee can advance a design for one problem which forecloses better solutions for another problem.

Thus the first goal of this paper is to provide a comprehensive look at floating-point, to provide sufficient understanding to evaluate current and future proposals. It covers not just what the dominant specification for floating-point, IEEE 754, says, but also what it *doesn't* say. It also covers the existing landscape of hardware and compilers do and don't do with regards to floating-point, including the switches all C++ compilers today provide to let users choose among varying floating-point semantics. Then it covers the choices made (or not made) by other language specifications, as well as covering the specific scenarios that need more specification in the current standard.

At this version of the paper, a full proposal for fixing the semantics is not yet provided. Instead, there is an exploration of the solution space for various aspects of floating-point semantics, and the author's personal preference for the which solutions work best in various scenarios. Individual scenarios can be progressed in future versions of this paper in omnibus paper, or split out into separate papers (some of which are already advancing independently).

§ 3. Background

To understand the specific problems with the current floating-point semantics in C++, one needs to have some background information on what even constitutes the semantics of floating-point, and in particular, what the differences are between the different options.

§ 3.1. IEEE 754

For most programmers, their first introduction to floating-point will be via IEEE 754: if a programming course touches on floating-point, it will generally introduce it by explaining as if all floating-point were IEEE 754. At the same time, it does need to be understood that while almost all modern hardware implements IEEE 754-ish semantics: there is often subtle variance from the exact IEEE 754 semantics. The *chief* problem of floating-point semantics is in fact in the need to pin down what the deviations from "standard" IEEE 754 are.

IEEE 754, also known as ISO/IEC 60559, is the main base specification of floating-point. While it defines both binary and decimal floating-point in the same document, since C++ does not (and is not actively looking to) support decimal floating-point, only the parts that are relevant to binary floating-point are discussed here. The main things that are provided:

- A general specification of floating-point formats, based on the radix (fixed to 2 for binary floating-point), the maximum exponent and the number of digits in the significand. Note that the minimum exponent and exponent range are inferred from the maximum exponent, as $e_{\min} = 1 - e_{\max}$ is required by the specification.
- The set of valid values for each floating-point format. This includes as special values $+0.0$, -0.0 , a set of subnormal (sometimes called denormal) values where the leading bit is 0 instead of 1, positive and negative infinities, qNaN (quiet not-a-number), sNaN (signaling not-a-number). The qNaN and sNaN may have multiple distinct representations, as they contain observable payload and sign bits, but which of those representations is chosen is generally left unspecified.
- The encoding of these floating-point formats into binary. This is generally the most well-known part of the specification, and "supports IEEE 754" is often colloquially used to mean "values are represented using IEEE 754 encoding" instead of "follows IEEE 754 specification rules precisely."
- Specific formats are given for 16-bit, 32-bit, 64-bit, and 128-bit floating-point formats, called binary16, binary32, binary64, and binary128 respectively, or "(binary) interchange formats" collectively.
- The concept of rounding modes, which are rules on how to convert the infinitely-precise result of an operation to the finite set of values allowed by a given format.
- The concept of exceptions. Unlike C++ exceptions, an IEEE 754 operation that raises an exception also returns a value at the same time, and exceptions need not interrupt execution. Indeed, the default behavior of floating-point exceptions is to set a flag and carry on execution.
- The concept of attributes, which are properties of blocks of code that change the behavior of floating-point operations statically contained in that block. In C, these attributes are mapped to pragmas, for example, `#pragma STDC FP_CONTRACT` controls whether or not an expression $(a * b) + c$ may be converted into `fma(a, b, c)`.
- A set of core operations and their behavior, especially with regards to special cases. The regular non-function operators, including arithmetic operators like `+` or `-` and cast operators both to integers and other floating-point types, are included in this set. Also included are `sqrt` and `fma`.
- A partial ordering of floating-point values is provided, and two sets of comparison operations are specified, one which raises exceptions on NaN inputs and one which does not. A recommendation on which version a source-level operator like `<=` should map to (although none is provided for a dedicated partial order operator like `<=>`).
- A set of recommended extra operations, which most of the functions in `<cmath>` correspond to. A side note is that IEEE 754 requires these functions be correctly rounded, but C does not, and most implementations do not correctly round them.
- A chapter on how programming languages must/should map source-level expressions to the underlying operations, which will be covered in detail in a later section of this paper.
- A chapter on what users need to do to expect reproducible results across diverse implementations.

In brief summary, IEEE 754 can be seen as providing reasonably well-defined semantics for the behavior of something like this:

```
enum class rnd_mode_t; // The rounding mode to use
struct fp_except_t; // A bitset of individual exceptions

// A pure operation, templated over an IEEE 754 format type.
template <typename T>
std::pair<T, fp_except_t> ieee754_op(T lhs, T rhs, rnd_mode_t rm);
```

There is a small amount of nondeterminism in this definition, for example, the payload of a NaN result is explicitly not constrained by the standard, and it does vary among hardware implementations. However, this nondeterminism can generally be ignored in practice, and it is probably not worth worrying about for a language specification.

§ 3.1.1. Non-IEEE 754 types

C++ compilers already support floating-point types that are not the IEEE 754 interchange formats, and so the standard does need to worry about such support. Many of these types are already IEEE 754-ish, and while they do differ from IEEE 754 semantics in sometimes dramatic ways, it is still generally safe to view them for the purposes of this paper as having something akin to the `ieee754_op` templates mentioned above.

`std::bfloat16_t`, while not an interchange format, is fully specifiable using the generic binary floating-point type rules for IEEE 754 (at least up until normal hardware variance), just using a different mix of exponent and significand bits. Indeed, it is likely that the next revision of IEEE 754 will incorporate this type in its list of floating-point types.

A more difficult IEEE 754-ish type is the 80-bit x87 FPU type, here referred to by its LLVM IR name `x86_fp80` to distinguish it from other types used as `long double`. Though it contains 80 bits, it can be viewed as a 79-bit IEEE 754 type, with an extra bit whose value is forced by the other 79 bits. If that bit is set incorrectly, the result is essentially a noncanonical value, a concept which IEEE 754 provides, but is not relevant for any other type already mentioned. Noncanonical values are alternative representations of a value which are never produced by any operation (except as the result of sign bit operations, which are guaranteed to only touch the sign bit) and should not arise except from bit-casting of an integer type to a floating-point type.

Beyond these two types, that support all of the features of the IEEE 754 standard but aren't directly specified by the standard, there also exist several types that do not fully adhere to the standard.

There have been several proposals for 8-bit floating-point types along the IEEE 754 encoding rules, but several of these make deviations to reduce the number of representations of special values, and may combine or even outright eliminate the notions of NaN values and infinities.

Some pre-IEEE 754 architectures are still supported by modern compilers, and they may have floating-point types which similarly lack the special value categories of infinity, NaN, or subnormals. Examples of such types include the IBM hexadecimal floating-point types and the VAX floating-point types.

The `long double` type on Power and PowerPC (`ppc_fp128`, to use LLVM's name for it), also known as double-double, is a more radically different floating-point type, consisting of a pair of `std::float64_t` values, the second of which is smaller than the first. Unlike all of the aforementioned types, this type is hard to describe via a sign-and-exponent-range-and-fixed-number-of-significand bits model, as the number of significand bits can change dramatically (consider the pair `{DBL_MAX, DBL_MIN}`), which would have a very large number of implied 0 bits in the middle of its significand).

Despite the diversity in the formats of non-IEEE 754 types, the concept of a well-defined, pure function implementing their fundamental operations that is templated on the type remains sound, and many of them even retain structures that correspond to the rounding mode and floating-point exception elements in the function signature. The diversity primarily impacts the definition of type traits in `std::numeric_limits`, and in the behavior of special cases, neither of which are being changed in this paper, and so we will consider the interface of `ieee754_op` to be sufficient for these cases, even if the implementation is very different.

§ 3.2. Hardware implementation

While most hardware is based on IEEE 754, very rarely is access to the pure `ieee754_op` interface described available to a compiler, essentially being limited to software floating-point implementations. Instead, most hardware chooses to define their core instructions by means of some sort of floating-point control/status register (FPCSR):

```
struct fp_env_t {
    // Contains several bits, these are some of the more common ones
    rnd_mode_t rm;
    fp_except_t flags;
    fp_except_t except_mask;
    bool flush_denormals;
};

// Global register
thread_local fp_env_t fpcsr;

// Parameters/return value correspond to FP registers
// Will call ieee754_op<T>, but will do other things based on fpcsr
template<typename T> T hardware_op(T lhs, T rhs);
```

Most commonly, there is the dynamic rounding mode in the FPCSR, which gets used as the rounding mode parameter of `ieee754_op`. Generally, exceptions will either cause a hardware trap, if it is instructed to do so by one of the bits in the FPCSR, or it will instead set a bit that can be read later (in C/C++, the appropriate test method is `fetestexcept`).

Additionally, there are usually other bits that control the actual semantics of the underlying operation. The most common of these is denormal flushing, but the precise behavior of denormal flushing varies between architectures. Some architectures even implement `hardware_op` in such a way that denormal flushing happens unconditionally, with no way to get the correct IEEE 754 behavior on denormals. Sometimes, the bits get more exotic: x87 has bits to control the effective precision, for example.

Some architectures provide the ability to specify the rounding mode statically on the instruction (i.e., pulling it from what is in effect a template parameter rather than the current value in the FPCSR), but this is by no means universal.

Some architectures, especially in the accelerator space, choose to just drop the concept of an FPCSR entirely, providing no means to maintain a dynamic rounding mode (or even a static rounding mode at times), or to observe floating-point exceptions.

Finally, it is worth noting that some hardware will have multiple FPU implementations, with the capabilities of those units diverging quite wildly, and sometimes using entirely different registers as their effective FPCSR. For example, x86 processors have both an x87 FPU (with relatively unusual semantics) and an SSE execution unit, which works more like typical FPUs. Since these units tend not to support the same sets of types (especially when SIMD vector types are accounted for), that means that the hardware capabilities can be at least partially type-dependent.

§ 3.3. Compiler implementation

Because of the need for optimizations on floating-point code, the internal representation of a compiler contains its own layer of semantics which is fairly independent of both hardware and language specifications. Indeed, to support the configurability of semantics via command-line flags, pragmas, or other such mechanisms, there is actually typically a very large number of variants for the floating-point semantics.

At a very high level, the compiler representation of floating-point semantics tends to fall into three main buckets of floating-point model. The first model is the one that demands a complete adherence to IEEE 754 or hardware semantics, including modelling the interaction with the floating-point environment fully and correctly: this is the `strict` model. The second model requires strict adherence only for the values being produced, and presumes that the floating-point environment is left untouched in its default state and no one is going to attempt to read the flags: this is the `precise` model. The final model goes further and allows the results of operations to vary in precision somewhat, and this is some kind of `fast` model.

Note that these models are buckets of actual semantics; in practice, the knobs of control within the compiler internally, made accessible via flags and other user-visible tools, can be tuned much more finely. There's a full combinatorial explosion of possibilities here.

For example, within the LLVM optimizer of the Clang compiler, there are the following flags that can be attached to a regular floating-point operator that can all be *independently* applied:

- 7 fast-math flags per instruction
 - `nnan`, which makes the use of NaN values in the operation undefined behavior
 - `ninf`, which makes the use of infinities in the operation undefined behavior
 - `nsz`, which allows `-0.0` to be represented as `0.0`
 - `arcp`, which allows `a / b` to be converted to `a * (1.0 / b)`
 - `reassoc`, which allows (among other things) reassociability to be inferred
 - `contract`, which allows FMA contractions to take place
 - `afn`, which allows some lower-precision approximations to be used
- 11 function attributes
 - `strictfp`, which prohibits speculation or reordering of floating-point operations
 - `denormal-fp-math`, which controls assumption of denormal flushing behavior
 - `denormal-fp-math-f32`, similar to above, but allows a different denormal flushing behavior to be chosen only for `float` values
 - `reciprocal-estimates`, which controls the number of refinement steps needed for an approximation of division or square root computations
 - A function-level version of each fast-math flag
- Optional instruction metadata indicating accuracy of result in ULPs.
- Variants of instructions
 - Constrained intrinsics of regular operations, with one extra parameter indicating assumed rounding-mode behavior, and another extra parameter indication whether or not floating-point exceptions may cause traps
 - Intrinsic versions of math functions, which must not set `errno`
 - Special intrinsic that can be expanded either to a single three-argument fma instruction or a pair of multiply and add instructions

Even so, this list is known to be missing variants that are necessary. It is likely that LLVM and Clang will add yet more fast-math flags in the future. The existing repertoire is deficient in supporting static rounding mode instructions, as well as supporting low-precision approximations (which are especially useful on offloaded code). Furthermore, several features may be removed entirely: the constrained intrinsics and `strictfp` operation are likely to be replaced in the near future, and the current handling of denormal flushing is problematic.

In general, any fixed list of relevant properties for optimization should be avoided: they are likely to change, both in additions and removals of parameters that influence optimization.

§ 3.4. Programming language semantics

At the end of this list of semantics is the compiler front-end, which needs to work out which of the variety of slightly-different shades of operations provided by the middle-end to map a source-level `operator+(T, T)` to. This choice is dictated both by the rules of the standard and the panoply of command-line flags or other language extensions (such as pragmas) offered by the compiler specifically to influence the choice of how to lower a language-level floating-point operation to internal IR and ultimately the final hardware instructions.

It is also important to note that the front-end is a completely different part of the compile from the optimizer. If the optimizer has a choice of whether or not it may make a transformation (which is true for most of the attributes mentioned in the previous section), the front-end is not generally capable of knowing if it will or will not make that choice. Most importantly, this means that the constant expression evaluation done by the front-end is done by an entirely different process than the constant folding done by optimizations, and it is not possible in general to guarantee that the two come to the same decision (in particular for things like contracting expressions into FMAs). C++ today does not guarantee equivalence between constant expression evaluation and runtime evaluation, and it is unlikely that implementations *could* make that guarantee.

§ 3.5. IEEE 754 rules for language specifications

IEEE 754, as mentioned earlier, has a small section laying out how programming language standards are to map expressions to the underlying operations. It only directly governs the rules for the behavior of an *individual* operation; an expression might comprise multiple operations, and most of that behavior is left up to the language specification. C++ already fulfills the basic requirements of defining types for intermediate values in expressions, specifying the order of operations.

The core requirements that are actually "shall" requirements relate to assignments, requiring:

implementations shall never use an assigned-to variable's wider precursor in place of the assigned-to variable's stored value when evaluating subsequent expressions

Similar language is used for the parameter and return types of a function: in all of these cases, IEEE 754 is explicitly precluding the use of extended precision of any kind. These rules are why C specifies `FLT_EVAL_METHOD` in the manner that it does, and even C++ alludes to this requirement in the footnote attached to `[expr.pre]`⁶.

Beyond these requirements are a few "should" recommendations. IEEE 754 envisions that the behavior is governed by "attributes," which are implicit parameters to the underlying operations. The main recommended attributes are an attribute to control the preferred format for intermediate values of an expression, and another attribute to control whether or not "value-safe optimizations" are allowed. Proposing ways in C++ to define attributes is one of the main goals of this paper.

Value-safe optimizations are more commonly known to users as fast-math optimizations. But even if value-safe optimizations are fully disabled, bit-exact reproducibility is not guaranteed. Properties like the sign and payload of NaN values need not be preserved by a value-safe optimization. Nor is the number of times (so long as it is at least one) or order of floating-point exceptions. However, things like the sign of 0, the exponent of a decimal floating-point number, or the distinction between an sNaN and a qNaN are not allowed to be changed by a value-safe optimization.

§ 4. Comparison of language standards

Floating-point is rarely described in detail by programming language standards, with most of them largely being silent on the sorts of issues described in this paper. What follows is a brief summary of the detail provided by other languages with regards to floating-point.

- Ada allows users to define their own floating-point types with minimum precision and optional range parameters in lieu of providing standard floating-point types most languages do (although there is a default `Float` type, which GNAT appears to map to IEEE 754 single-precision). The specification has two modes for floating-point, a strict mode which requires strict conformance to the floating-point model of the standard, including raising overflow errors if a computation overflows; and a relaxed mode which allows some of the requirements to be relaxed (in particular, forgoing overflow errors on floating-point overflow). The GNAT compiler provides two separate floating-point pragmas here, one which turns on the overflow check, and one which is roughly equivalent to C's `#pragma STDC CX_LIMITED_RANGE ON`. Intermediate range checks, for floating-point types that declare limited range, are still required even in relaxed mode.
- C has arguably the single most detailed discussion of floating-point in its reference manual. However, a lot of that detail is locked behind the implementation-dependent Annex F, and many implementations (even when claiming conformance) fail to fully conform. An Annex F-conformant compiler is still allowed to engage in excess precision and contract expressions, subject to some restrictions that are poorly observed in practice. Denormal flushing is by inference prohibited in Annex F, but may be allowed outside of it, given the general latitude allowed by the standard. Full sNaN support is not required, even in Annex F. The environment requires use of a pragma to access, and whether or not an expression occurs at runtime or compile-time (for purposes of environment influence) is strictly controlled by the standard. A few macros are provided by the compiler/runtime to indicate the capabilities of the floating-point types, but these macros do not, in practice, reflect the behaviors when compiled with fast-math.
- C# requires the IEEE 754 formats for its floating-point types, but explicitly allows excess precision to be used. Additionally, it explicitly mentions that denormal support is optional, neither requiring nor forbidding it.
- Forth provides IEEE 754 single-precision and double-precision floating-point types, in addition to an implementation-defined floating-point type. The general rounding behavior is implementation-defined, and there is no further discussion of other situations discussed in this paper.
- Fortran behaves like C in many ways with respect to floating-point. The language predates IEEE 754, and so IEEE 754 support is optional. An intrinsic module is provided that allows querying which types are IEEE 754-compliant, with

compliance here being a set of rather loose properties. For example, the base `IEEE_SUPPORT_DATATYPE` doesn't require complete conformance with IEEE 754, but only that the format for normal numbers matches an IEEE 754 base datatype, that one of the rounding modes matches for regular arithmetic, and that a few particular IEEE 754 operations are provided. There are also queries for support for NaNs, infinities, and subnormal numbers, but experiments show that these are not affected by fast-math flags. Additionally, routines are provided for querying or modifying the floating-point environment, including rounding mode, exceptions, rather exceptions trap, and enabling or disabling denormal flushing; the behavior of the environment when the intrinsic modules are not used is akin to the way C supports `#pragma STDC FENV_ACCESS OFF`. Finally, expressions are allowed to be substituted for mathematically-equivalent expressions, except across parentheses: this allows some non-value-safe optimizations (such as FMA contraction) to occur. There is no direct mention of excess precision concerns.

- Go explicitly maps its types to the IEEE 754 format types. FMA contraction is explicitly permitted, even across statements, with only an explicit type conversion providing a contraction barrier. Otherwise, no mention is made of floating-point environment, denormal flushing, rounding mode, or excess precision.
- Java, starting from Java 17, requires strict adherence to IEEE 754 standard, prohibiting any optimizations that would be a value-changing optimization. Denormal support is explicitly required. From Java 1.2 to Java 16, there was a non-strict mode which allowed excess exponent range (but not number of mantissa bits) to be used for floating-point values; code could opt out of this mode by using the `strictfp` keyword on methods. This nonstrict mode is not quite equivalent to the situation where `FLT_EVAL_METHOD == 2` in C.
- JavaScript uses an IEEE 754 double precision type as its main numeric type and has an unusually descriptive abstract machine that explicitly references the behavior of IEEE 754 default rounding mode in its description of core arithmetic operations. Some functions, such as `Math.cos`, have explicit license to be approximated.
- Julia has support for the 16-bit, 32-bit, and 64-bit IEEE 754 floating-point types. These are expected to generally follow IEEE 754 semantics. For supporting fast FMAs, it provides an operation `muladd` which is defined to be the faster of FMA or multiply-and-add. Functions exist to change the rounding mode and denormal-flushing modes globally, even changing the floating-point environment of other flags. Finally, it provides a macro facility that enables LLVM's fast-math flags on a per-operation basis.
- Kotlin provides IEEE 754 single-precision and double-precision floating-point types, but defers details of their implementation to the implementing platform. The main platform used for Kotlin is the JVM, so similar rules to Java can be inferred to apply.
- Lua has a float type, which can either be mapped to a `float` or a `double`, depending on how the interpreter was compiled. While not explicit in saying so, it can be inferred that the semantics follow the semantics of the C type on the host machine.
- MATLAB provides several different floating-point types, including a full arbitrary-precision type, although the default it uses is the IEEE 754 double-precision floating-point type. The language reference largely does not discuss the topics here, although as an array-based language with thread-based, offload, and distributed parallelism modes, several array language constructs tend to have explicit footnotes saying that their precision isn't guaranteed. Furthermore, the reference does link to the blogs of some MATLAB developers who discuss some of the topics in this paper, with the general inference being that MATLAB is not going out of its way to provide reproducible results across diverse hardware.
- Pascal provides a single `real` type to represent floating-point types, although several implementations do provide extensions that cover the major IEEE 754 formats. The result of `real` arithmetic operations are, per the standard, "approximations to the corresponding mathematical results" whose accuracy is implementation-defined.
- Perl has a single floating-point type, which the documentation implies is the underlying `double` type on the host architecture.
- PHP has a single floating-point type, which the documentation implies is the underlying `double` type on the host architecture.
- Python uses the underlying representation of `double` in its implementation (either C or Java) for floating-point types, and says explicitly that "you are the mercy of the underlying machine architecture" for behavior here.
- R does not explicitly mention anything with regards to IEEE 754 conformance.
- Ruby uses the underlying `double` type on the host architecture.
- Rust does not yet officially document its precise float semantics guarantee yet, but a recent accepted proposal lays out the guarantees it intends to support. Rust wants to mandate strict IEEE 754 conformance, without support for floating-point environment, but it acknowledges buggy implementation on some platforms with regards to excess precision, sNaN handling, and denormal flushing. Uniquely among the languages considered here, Rust actually provides some

guarantees on the behavior of NaN payloads, essentially guaranteeing that the compiler will not introduce certain payloads if neither the source nor the hardware do so.

- Swift explicitly uses IEEE 754 formats for its floating-point types, but does not go into any further details on the semantics of floating-point.

§ 5. Motivation

Floating-point semantics in C++ are well-known to be thoroughly underspecified. In recent years, though, there is a resurgence of interest in bringing clarity to the specification. The goal of this paper is to provide a comprehensive look at what needs to be done to clarify the semantics, as partial solutions that only tackle a subset of concerns may generalize poorly to the full problem space.

This motivation section is split into two subsections, looking at the existing problems from two different perspectives. The first section will focus on implementers and the varying hardware semantics and compiler models they have to support. The second section will focus on users and on specific use cases that they might want to achieve.

§ 5.1. Implementers' perspective

§ 5.1.1. Excess precision

This is a subject that has come up recently in a few papers. Most prominent are P3565 and P3488.

The core problem that `FLT_EVAL_METHOD` tries to solve is the x87 problem. The x87 floating-point unit only supports internal computation with one data type: the 80-bit floating-point type that compilers targeting it map `long double` to. It lacks any arithmetic support for 32-bit and 64-bit floating-point values, although the unit has load and store instructions for such types that convert to/from the 80-bit type as appropriate.

Unlike integer types, it is not always the case that a smaller floating-point type can be losslessly emulated by using larger floating-point types; the larger type needs to be sufficiently larger to avoid double rounding (for more details, see [this academic paper](#)). For the standard IEEE 754 sequence of types (binary16, binary32, binary64, binary128), it is the case that each type can be emulated with the next one in the sequence without risk of double rounding. But this is not the case for the x87's 80-bit type: it cannot emulate IEEE 754 binary64 arithmetic without inducing double rounding.

To solve this problem, C99 added `FLT_EVAL_METHOD`, which allows an implementer to evaluate the temporary values within expressions in higher precision instead of strictly sticking to the exact source types. However, at prescribed points in the program (when assigning to a variable, use as a parameter or return value, or using an explicit source cast), the value must be truncated to its source type.

Despite the presence of this feature, most modern compilers do not in fact correctly implement the behavior required of `FLT_EVAL_METHOD == 2`. Instead, the compiler frontends lower the code to an IR where all the values are using the lower-precision binary32 and binary64 values, and merely map the IR's implementation of `operator+(binary64, binary64)` to the hardware `FADD` instruction. The following table illustrates the consequences of this difference in implementation (using LLVM IR as the representation for a generic compiler's internal IR):

IR		
Incorrect (implemented by clang, gcc)	<pre>define double @do_add(double %a, double %b, double %c) { %sum1 = fadd double %a, %b %res = fadd double %sum1, %c ret double %res }</pre>	<pre>do_add: fld qword ptr [esp + 20]; lo fld qword ptr [esp + 12]; lo fld qword ptr [esp + 4]; lo faddp st(1), st ; ad faddp st(1), st ; re ret ; (r</pre>
Correct (implemented by icc)	<pre>define double @do_add(double %a, double %b, double %c) { %a.conv = fpext double %a to x86_fp80 %b.conv = fpext double %b to x86_fp80 %c.conv = fpext double %c to x86_fp80 %sum1 = fadd x86_fp80 %a.conv, %b.conv</pre>	<pre>do_add: sub esp, 12 ; re fld qword ptr [esp + 32]; lo fld qword ptr [esp + 24]; lo fld qword ptr [esp + 16]; lo</pre>

```

%res = fadd x86_fp80 %sum1, %c.conv
%res.conv = fptrunc x86_fp80 %res.conv to double
ret double %res.conv
}

```

```

faddp st(1), st ; add
faddp st(1), st ; re
fstp qword ptr [esp] ; st
fld qword ptr [esp] ; lo
add esp, 12 ; re
ret ; (r

```

Incorrect behavior can be observed in other ways. For example, a sufficiently large floating-point expression that requires spilling intermediate results due to insufficient registers causes those results to be spilled as their source types rather than the correct extended precision types. Storing a result in a variable fails to force truncation of the extended precision arithmetic. As a result, the actual semantics implemented by these nonconforming compilers amounts to evaluating all `float` and `double` arithmetic in extended precision, except that at unpredictable points in the time, it is truncated to the source precision. This behavior is not helpful for users, since there is little or no ability to influence the actual truncation behavior.

That compilers do not conform to the correct behavior is long-known. The [gcc bug](#) pointing out the issue for x87 is the second-most duplicated bug in its bug tracker (eclipsed only by the bug used for reports based on alias violations in user code). If 25 years and 100 duplicates is not enough to motivate a compiler to make their code conforming, then there is little hope of the compiler ever doing so. Clang similarly has a [long-open bug](#) on its nonconformance here, and while there is discussion on how to fix it, it is not considered a priority.

The problems described here are relevant for very few architectures. For x86 processors, the SSE and SSE2 instruction sets added an IEEE 754-compliant implementation for binary32 and binary64. The last x86 processor released without SSE2 support was in 2004, and the 64-bit ABIs all require SSE2 support, which means only x86 processors targeting 32-bit applications and supporting hardware 20 years old cannot easily conform to precise floating-point semantics for binary64. Outside of x86, the next most prevalent architecture that has the excess precision problem is the Motorola 68000 family, where FPU's before the MC68040 (released in 1990) lack the ability to do binary32 or binary64 arithmetic exactly.

Given the declining importance of architectures for which a solution like `FLT_EVAL_METHOD` is necessary, and given that current compilers largely do not conform to the specification where it is relevant, the most prudent course of action is to not reserve any space in the standard for these implementations and accept that compilers will likely always be non-conforming on these architectures.

5.1.2. Denormal flushing

For various reasons, many hardware implementations have opted to not implement proper support for denormals, sometimes providing an option to opt out of denormal support via a bit in the floating-point environment, or sometimes even going so far as to provide no mechanism to support denormals at all. As a result, for some architectures (such as the original ARM NEON implementation), flushing denormals is necessary to be able to use hardware floating-point at all.

Some hardware supports denormals only via expensive microcode or software trap handling for the denormal cases. For an individual instruction, the execution penalty for a denormal input can be 100 times slower. Averaged over an entire benchmark (which obviously executes more than just floating-point instructions involving only denormals), this tends to be single-digit percentage loss or less, unless the compiler believes it is necessary to flush denormals to be able to access a vectorized SIMD unit. However, full-speed hardware with full denormal support is known now, and many architectures that previously required denormal flushing, or imposed severe speed penalties on denormals, are able to do use denormals with no speed impact on their newest versions. Thus, denormal flushing is also an issue whose salience is decreasing and will become less of an issue in the future.

A main complication of denormal flushing is that some implementations choose to link in a library that sets the denormal flushing bit in the environment on startup when linking with fast-math flags. Owing to user complaints, this has shifted recently to linking in this library only when compiling an executable and not a shared library. Consequently, whether or not denormals will be flushed is unknowable by the compiler as it compiles a translation unit. In such implementations, a `constexpr` function indicating support for denormals can only be at best a guess and cannot be made reliable.

5.1.3. Associativity and vectorization

It should be fairly well-known that floating-point arithmetic is nonassociative, which means `a + (b + c)` may return a different result from `(a + b) + c`. Unfortunately, associativity is a required property for parallel algorithms, so the nonassociativity blocks the ability to automatically block code. All C++ compilers provide some means to allow assumption

of associativity to enable vectorization. Frequently, this also allows the related assumption of distributivity (allowing $a * (b + c)$ to be converted to $a * b + a * c$ or vice versa).

For most numerical code, these are generally safe assumptions to make. If all of the values involved are about the same magnitude and the same sign, then the resulting value of the expressions will only differ in the last few bits of the significand, a difference subsumed by the inherent inaccuracy of the source data in the first place. When signs are different, there is the potential for values to be greatly different due to overflow (if $b + c$ is positive infinity and a is negative, then $a + (b + c)$ would be infinite where $(a + b) + c$ could be a finite value), or other artifacts due to catastrophic cancellation.

There are times when these assumptions are not safe. Some algorithms rely on the precise order of arithmetic to get extra precision. For example, Fast2Sum and Kahan summation provide extra precision that is destroyed with reassociation:

```
// Return two values such that sum + error is the exact result of a + b, without
// any precision loss.
std::pair<double, double> fast2sum(double a, double b) {
    double sum = a + b;
    // With reassociation, the compiler would turn this into double error = 0.0;
    double error = (sum - a) - b;
    return {sum, error};
}

// Return a more precise estimate of the sum of the values than naive summation
// would give.
double kahan_summation(std::valarray<double> vals) {
    std::pair<double, double> sum = {0, 0};
    for (double v : vals) {
        sum = fast2sum(sum.first, v + sum.second);
    }
    return sum.first;
}
```

§ 5.1.4. FMA contraction

Many, though not all, hardware floating-point units offer an FMA instruction, that computes the value $a * b + c$ in a single step, without any intermediate rounding. The resulting instruction is usually faster than doing the operation as separate instructions, and usually the extra precision is more helpful for the user (though there are times when it is better to do it as two separate instructions). Converting the source expression in this manner is known as contraction, and almost all contraction in practice tends to either be to an FMA instruction or some instruction that differs only in the signs of the inputs.

As the FMA operation is one of the core operations mandated by IEEE 754, there is practically always an implementation of FMA available, even if the hardware lacks such an instruction. However, the emulation of FMA in software for such hardware is slow, and many users would rather use the two-instruction multiply-and-add form if that is the faster alternative.

Given the utility of FMA contraction, several languages do provide guidelines for FMA formation. C provides a `#pragma STDC FP_CONTRACT ON` facility, that allows contraction within expressions. This is subtly different from the compiler flag (or equivalent pragma) provided by many compiler implementations, which will contract across expressions as well. Fortran provides a general expression rewriting ability which includes FMA contraction.

From the perspective of an optimizer, an operation `fast_fma` whose semantics are "do an FMA operation unless an `fmul-` then-`fadd` is faster" turns out to be easier to work with. The code would start out as a single operation and remain as a single operation throughout the entire optimization sequence, with little risk of an optimization moving only part of the operation to another location (e.g., hoisting out of a loop); it is also easier to reason about which version is desired by the user for the purposes of constant folding or evaluation. Additionally, representation for two operations increases the risks that other optimizations end up deleting optimization barriers that would have prevented undesirable formations of FMA.

The big problem with a `fast_fma` approach, however, is that it is a ternary operation and more cumbersome to use as an operator in otherwise typically infix code, especially given that there exists a readily available syntax for the operation via common operators (namely `*` and `+`). Furthermore, some users may object to having to add extra methods to overload to make their custom number-like types work well.

Finally, it should be noted that FMA contraction is not always a good thing, even on hardware where it is known to be fast. The expression `a * b + a * -b`, if evaluated as two multiplies and an add is guaranteed to be exactly `+0.0` so long as the inputs are finite. But if it is evaluated with a multiply and an FMA, then it is likely to be a small value. Similarly, there are expressions where evaluation via solely multiplies and adds would guarantee the result to be positive, but if done via FMAs, it could be negative depending on the vicissitudes of rounding. Consequently, while it may be desirable to turn on FMA contraction by default, it is absolutely necessary to retain the ability to disable it for code that doesn't want it.

5.1.5. Fast-math

In general, fast-math optimizations are any floating-point optimization that would be mathematically equivalent if the numbers were real numbers, but are not equivalent for floating-point expressions. Reassociation and FMA contraction, as discussed above, are two such optimizations, but there exist other ones that are not worth calling out into a separate section. These optimizations tend to fall into two buckets.

The first bucket of fast-math optimizations are ones that ignore the existence of the special floating-point values: negative zero, infinities, and NaNs. For example, the expression `x + 0.0` is equivalent to `x` for all floating-point values save `-0.0` (as `-0.0 + 0.0` is `+0.0`). Just as unlikely integer overflow can impede certain optimizations, the unlikely presence of these special values too impede the ability to do some basic arithmetic optimization; fast-math flags allow users to opt into these optimizations when they can guarantee they will not be intentionally using these special values. It should be noted that there is vociferous disagreement as to whether or not `std::isnan(nan)` should be considered undefined or not when fast-math is in effect.

The second bucket of fast-math optimizations are ones that do not preserve the precision of the resulting values. In addition to the optimizations discussed in previous sections (which are all of this category), another common example is being able to convert `a / b` into `a * (1.0 / b)`, with the reciprocal expression hopefully being able to be hoisted out of a loop. Or one can convert `pow(x, 0.5)` to `sqrt(x)` (although note that `pow(-0.0, 0.5)` is `+0.0` while `sqrt(-0.0)` is `-0.0`).

5.1.6. Constant expressions

In a strict floating-point model, the environment of floating-point operations is important, and consequently, it matters a great deal whether or not a given operation is to be evaluated at compile-time or at runtime. Here, the definition of "compile-time" is specifically constant expression evaluation within the frontend: the constant folding that may be done by an optimizer merely has to preserve the illusion that it is done at runtime, and so long as the code initially generated by the frontend has annotations that the operations interact with the floating-point environment, that property is relatively easy to uphold in the optimizer.

When implementing C's Annex F rules for floating-point environment, the guideline for whether a given floating-point expression is evaluated at compile-time or at runtime is clear: the initializer of an object with static or thread storage duration is done at compile-time, while everything else must be done (as if) at runtime. Of course, C lacks the `constexpr` machinery of C++, and thus there is very little opportunity to do interesting stuff at compile-time, making such a simple rule easy to apply. C++ requires applying more careful analysis.

The most natural extension of C's rules here is to say that any expression that is part of a *core constant expression* needs to occur (as if) at compile-time; any floating-point environment effects that are observed there would not be observable in the program. Furthermore, any floating-point expression not part of a core constant expression occurs (as if) at runtime. Thus, if the expression is such that `std::is_constant_evaluated()` would return `true`, the user could expect that the code will definitely be executed at compile time; and if it would return `false`, they would know that the effects would be visible to runtime functions that manipulate the floating-point environment.

Another issue with constant expressions in C++ is the role of environment during constant expression evaluation. Since C++ allows for the ability to have statements with side effects in constant expressions, it is possible to specify that functions effecting the floating-point environment do so in constant expressions as well, although it may not be desirable to do so.

A final issue is that adjustments like fast-math optimizations are unlikely to be implemented the same in the constant expression evaluator as they are in the optimizer or the runtime evaluation. For example, if FMA contraction is enabled, the constant expression evaluator generally has no way of knowing if the runtime optimizer is capable of contracting the expression, and it is unlikely to match. Constant expression evaluators today tend not to adapt to the current fast-math flag state during constant expression evaluation.

5.1.7. Type traits

C++ provides a few classes of type traits to indicate the properties of floating-point types and their arithmetic operation. One of the issues with these traits is that their interpretation is not fully clear in the presence of fast-math optimizations, especially given that the ability to turn such optimizations on for a finer-grained scope means that whether or not they are in effect may change throughout a single translation unit.

The most concrete example is to look at `std::numeric_limits<float>::has_quiet_NaN`. In the case of a fast-math mode that makes use of NaN values undefined behavior, should this value return true or false? At present, all implementations return `true` for this statement, which means that the behavior reflects whether the *format* supports qNaN values rather than whether the computation actually supports it meaningfully. Similar behavior can be observed for the meanings of `is_iec559` (which, in practice, amounts to "is this IEEE 754-format" and not "does this obey IEEE 754 arithmetic" rules).

In principle, it's possible to add methods to query the adherence to fast-math behaviors. Clang and GCC already provide macros like `__FAST_MATH__` that are defined in fast-math mode. However, these macros similarly don't capture the behavior in place for a scope, only the request at the command-line. Furthermore, as fast-math is a collection of individual properties, it's not immediately clear what the value should be if only some of the fast-math optimizations are enabled. Replacing these macros with special standard library functions is generally inadvisable because either the functions would return incorrect results due to differences at the point of evaluation or it would require a lot of machinery that doesn't exist in compilers today.

5.2. Users' perspective

Several of the issues mentioned above are also issues that matter to users (in particular, fast-math is often motivated by users' desires rather than implementers' whims), but there are a few issues which tend to be dominated by the need of users to do particular things.

5.2.1. Reproducible results

One of the main concerns for some users is the need to reproduce results that are identical across a diverse array of platforms. This is particularly salient in the video game industry, where slight variances can cause multiplayer games to desync (fail in such a way as to cause players to be kicked out of the game). While most numerical code tends to already be built on a general assumption of a mild degree of inaccuracy already and can thus tolerate some degree of deviation among diverse implementations, there are times when particular sequences are exactly needed (e.g., in Kahan summation, as mentioned above), and thus defense from a sufficiently smart compiler is necessary.

Irreproducibility arises from several sources:

- Hardware doesn't need to implement IEEE 754-based arithmetic. However, in practice, most hardware that people target does support at least the IEEE 754 formats, so this isn't a major source in modern times.
- The behavior of IEEE 754 is underspecified in a few cases, most notably the handling of NaN payloads and the definition of tinyness for reporting underflow (although floating-point environment support is not relied on in most code anyways). IEEE 754 itself says that reproducible code should not rely on these behaviors anyways.
- Code compiling for the x87 FPU tends to want to use excess precision to avoid the performance penalty of emulating correct behavior on that FPU. Since this is a platform that has been obsolete for decades, it is declining in relevance for modern software.
- Fast-math optimizations, which include FMA contraction or reassociation, mean that the compiler has latitude to choose multiple variants when they are enabled and it should be expected that those choices differ from platform to platform.
- Denormal flushing may or may not be enabled on various platforms, and this can give different results. Also notable is that denormal flushing tends to partially rely on the definition of tinyness, so whether or not a given operation would be flushed to zero can itself change.
- The floating-point environment (which can include exotic flags like x87's precision control flag) may have been adjusted by other libraries linked into an application, and this environment can affect the results.
- Differences in vector width on different platforms can result in vectorized algorithms being reassociated differently. Similarly, distributing work to a different number of threads can result in a reduction being done differently.

- Approximate instructions (such as the x86 [RSQRTSS](#) instruction) can have different implementations by different vendors or even different microarchitectures of the same vendor.
- Standard math libraries do not have accuracy guarantees, and so the same compiler, linking to different standard libraries on different platforms, may produce different results for the same input. Libraries themselves may return different values for different library versions. Constant evaluation or constant folding may use the host library to evaluate these functions, and thus the result can vary based on the host platform of a cross-compiler, even given otherwise identical compiler and target platform.
- Math libraries such as BLAS routines may choose their kernels differently based on host parameters such as cache size, and thus minor variations in the chip may result in different values.

Most users cannot be expected to know all of the ways that their floating-point code is not reproducible. Thus, we need a feature that can reliably reproduce floating-point code, even in the face of compiler flags saying "please make my math irreproducible."

5.2.2. Rounding mode

The default floating-point model used by most compilers does not allow reliable access to the rounding mode or floating-point environment. As a consequence, these features tend to go unused by implementations, even where they might be helpful. Of the underused portions of the environment, the most useful is the rounding mode. Furthermore, there is a growing trend in modern hardware to add floating-point instructions where the rounding mode is an operand of the instruction itself rather than relying on the rounding mode specified in the floating-point environment, and it is useful to be able to have a language facility that more directly maps to this style of hardware.

5.2.3. Environment access

Being able to access the other bits of the floating-point environment are occasionally useful. Floating-point exceptions do indicate erroneous situations, after all, so being able to observe the error of individual operations is helpful in some cases, much as users will sometimes want to test whether an individual integer multiplication overflows. An example of some code that does this looks as follows:

```
float scale(float value, float pointScaleFactor) {
    // Ignore all previous exceptions that may have happened,
    // we just care about this one operation.
    feclearexcept(FE_ALL_EXCEPT);
    float result = value * pointScaleFactor;
    if (fetestexcept(FE_OVERFLOW | FE_UNDERFLOW)) {
        // report error ...
    }
    return result;
}
```

The sticky nature of floating-point exception makes it easy to support multiple operations or even entire numerical algorithms if that's desired, but it also does require clearing exceptions before doing the operations in question. Most hardware implementations also provide the ability to turn floating-point exceptions into traps, which could be combined with a software trap handler to do fine-grained reporting of floating-point error conditions with minimal overhead in the cases where conditions occur.

A parallel to atomic memory references can even be drawn with floating-point exceptions. In this model, what is generally desired is not that all floating-point operations and their associated exceptions occur strictly in accordance with the source behavior, but rather that they don't get moved across certain function calls. The calls to floating-point environment functions can be seen as similar to atomic fences.

§ 6. Solution space

Having covered in detail the existing issues, the next thing to turn to is the menu of options available to solve these problems. These options are not mutually exclusive, nor is it necessary to pick the same option for different features.

§ 6.1. Do nothing

It is always an option to not attempt to say anything about the precise details of floating-point semantics. This is what C++ largely does today, and as the survey of programming languages shows, many other languages are able to get by with only vague hand waves to behavior.

§ 6.2. Unspecified behavior

Explicitly unspecified behavior is another avenue for some of the semantics. In cases where some degree of nondeterminism is already expected, making the floating-point behavior itself be nondeterministic can provide a lot of benefit without adding much, if any, tax to the user's mental model. Indeed, today C++ leverages this in its definition of [GENERALIZED_SUM](#).

§ 6.3. Demand strict conformance

Demanding strict conformance to IEEE 754 arithmetic in all aspects is the extreme opposite of saying nothing. However, as already extensively detailed, compilers deviate from IEEE 754 in a myriad of small ways, and they are extremely unlikely to go for strict conformance just because the standard demands it of them. I judge it better for the standard to admit reality here and instead discuss how to cope with deviations from IEEE 754 than live in a pretense that everybody is strictly conforming.

§ 6.4. Pragmas

While the committee may look unfavorably on pragmas in general, it is worth bearing in mind that some times they are the most appropriate tool for the job. When it comes to controlling the semantics of floating-point operators, pragmas are by far the most common option chosen, with all of the languages that specify means for users to control their behavior doing so via pragmas are pragma-like equivalents (see section 4). Indeed, a majority of C++ implementations *already* support pragmas for some of these features (and even if other mechanisms are chosen, it is substantially likely that they will be implemented via existing pragmas).

The main advantage of pragmas as a tool is that they are infinitely generalizable. If a compiler decides to add a new knob to the floating-point behavior, it is trivial to add user support for that knob via pragmas.

Pragmas do have significant drawbacks though. They do not work well with generic code, since there is currently no way for code to declare that it needs to inherit the pragma state of its caller:

```
template <typename T> struct wrapper {
    T val;
    wrapper(T val) : val(val) {}
    // Given these function implementations ...
    wrapper<T> operator+(wrapper<T> o) { return val + o.val; }
    wrapper<T> operator*(wrapper<T> o) { return val * o.val; }
};

typedef wrapper<float> wfloat;

// ... this should compile down into an FMA...
wfloat use_fma(wfloat a, wfloat b, wfloat c) {
    #pragma STDC FP_CONTRACT ON
    return a * b + c;
}

// .. but this one shouldn't...
wfloat dont_use_fma(wfloat a, wfloat b, wfloat c) {
    #pragma STDC FP_CONTRACT OFF
    return a * b + c;
}

// ... but the pragmas can't reach into the operator function definitions!
```


There are some extensions that might be able to mitigate this problem of inheriting floating-point context. One can imagine an attribute that would indicate that the function does so:

```
// In addition to inheriting floating-point context, this would also signal the
// equivalent of always_inline and forbid taking the address.
// NOTE: this does violate standard attribute ignorability rules.
[[intrinsic]] wrapper<T> operator+(wrapper<T> lhs, wrapper<T> rhs) {
    return lhs.val + rhs.val;
}
```

Or a template parameter that can inherit the floating-point context:

```
// Special parameter value that inherits the context of pragma state from its
// caller context.
template <float_context ctx = inherit_float_context>
wrapper<T> operator+(wrapper<T> lhs, wrapper<T> rhs) {
    return lhs.val + rhs.val;
}
```

§ 6.5. Attributes

C++ attributes can attach to blocks and function definitions, which provide sufficient functionality to do what the C floating-point pragmas do while avoiding use of the preprocessor entirely. The rule that standard attributes have to be ignorable limits their use to only controlling those floating-point features that resemble fast-math flags rather than those that are making the behavior stricter.

§ 6.6. Fundamental types

Another avenue of exploration is augmenting the floating-point types to represent varying floating-point semantics. These augmentations can come in the form of new types (similar to how `std::float16_t` were added), in the form of custom type specifiers and/or qualifiers, or in the form of standard library templated types (discussed further in the next section).

The primary advantage of representing floating-point semantics in this way is that it tends to compose well with the use of templates for generic code. Any code that needs to be generic over the precise floating-point semantics can easily do so just by templating over these types, without need for any other language features.

The primary disadvantage of this representation is that it is not composable with the highly tunable nature of floating-point semantics. Each knob creates a combinatorial explosion of new types to handle. Type specifiers might at least avoid the need to name each member, but they do not remove the need to provide a new version of the function for each member of the power set of qualifiers (or at least a new instantiation of a templated function).

A more subtle disadvantage is that this approach attaches the behavior of semantics to types rather than to operations themselves, and that makes the task of mapping an operation to its semantics--especially when the operation has heterogeneous types for parameters--more difficult not only for the implementer but also for the user (in their mental model). Special care would also have to be given to the behavior of implicit and explicit conversions for these types, and such conversions are already a problem for floating-point types which can have at least three distinct types representing the same underlying type today.

§ 6.7. Type wrappers

A commonly suggested approach for solving these problems is the use of templated type wrappers for floats, something like `fast_float<float>` or `reproducible<float>`. These share much of their trade-offs with the previous case of fundamental types, but they also have some interesting differences.

First, they move the semantics from core language to the library portion of the specification. In implementation terms, they still ultimately need some sort of secret handshake between the compiler and the standard library, but this can reuse existing

compiler features. It also allows them to be experimented with and tested without needing to use a custom compiler, making them an easier vehicle to gain implementation experience.

However, they also differ from something like qualifiers in that the syntax of of templates creates additional burdens for the high multiplicity of control knobs, which varies slightly depending on how these knobs are handled in template form.

One approach is to represent each knob as an independent template wrapper, for example `reassociable<T>` to enable reassociation, `contractable<T>` to enable FMA contraction, etc. This allows for a complete open set of properties--it's infinitely generalizable--but it is also prone to the problems that `reassociable<contractable<T>>` is a different type than `contractable<reassociable<T>>`.

Another approach is to use just one single template wrapper, and have template parameters for each possible knob, e.g., `template <typename T, bool contract, bool reassociate> struct fast_float`. This approach resolves the order of wrapping approach that independent wrapper types would have. But in the process, it makes the set of available options essentially a closed set.

A third approach is to use a single template wrapper and a single configuration parameter, but have the configuration parameter be a struct parameter and rely on designated initializers to make the specification of the template be somewhat tolerable for users:

```
struct fast_flags {
    bool nnan;
    bool ninf;
    bool nsz;
    bool reassoc;
    bool contract;
    bool afn;
    bool arcp;
};
template <typename T, fast_flags f> struct fast;
fast<float, { .nnan = true, .ninf = true }> fast_val;
```

§ 6.8. Free functions

As opposed to attaching the floating-point semantics to types, it is instead possible to attach them to functions themselves, for example, providing a `std::fast_fma` function that may optionally evaluate as one or two operations for the purposes of rounding.

The chief advantage of such an approach is that most of the semantic knobs tend to be oriented around the actual approach; unlike attaching to types, where there is a potential to mix heterogeneous semantic operands. Free functions also lend themselves to introducing new operations that aren't easily indicated the via current operators used in regular infix notation for C++ types, e.g., the FMA operation or Kahan summation.

The disadvantages of free functions is that they do not necessarily play well with custom wrapper types. Clever use of `if constexpr` can ameliorate this to a degree, allowing an implementation to call an overload of `fma` if it is available or otherwise falling back to `a * b + c`, but it still adds friction to the design of such libraries.

§ 6.9. Special lambdas

A final category of change is to have standard library functions that take as an argument a lambda whose body is compiled in a different mode. This has been used with a degree of success by SYCL, where offloaded kernels are indicated by this kind of mechanism:

```
cgh.parallel_for(1024, [=](id<1> idx) {
    // This body executes in an offloaded device context, not the host context.
});
```

The advantage of this kind of approach is that it creates a function call barrier between the code in the lambda body and the code outside of it, and function calls are very natural optimization barriers for a compiler. Furthermore, lambdas' ability to

capture their environment means there is relatively little writing overhead to moving code that needs to be protected into the lambda body.

The main disadvantage is that this does not work very well in contexts where the frontend needs to generate different IR for different floating-point contexts, since a compiler can easily only compile the lambda body, and not the functions it calls, in a different mode. Doing a call graph traversal to find recursively called functions to generate their bodies in a different mode is ill-advised in the frontend of the compiler, since it's generally going to be less accurate and will likely result in a slew of bugs where it misses various awkward implied calls.

§ 7. Proposal

This section gives a summary of the current state of C++ with respect to the issues mentioned in section 5, a discussion of how some of the existing issues might be fixed, and the author's proposed fixes, with rationale as to why.

§ 7.1. Floating-point formats

At present, `float` and `double` are not required to be IEEE 754 formats. It is possible to strengthen the specification to require them to follow the IEEE 754 specification as far as the layout is concerned. There is very little existing hardware which have hardware floating-point support but lack support for the IEEE 754 formats. Some microcontrollers do map both `float` and `double` to IEEE 754 single-precision format.

The main benefit to dropping support for non-IEEE 754 formats is that it makes it possible to omit consideration of types that lack infinities or NaNs for the purposes of special-case behavior in math functions. However, the current specification doesn't go into any detail here anyways, and the C specification's discussion of various kind of issues is sufficient to cover this, if it were adapted into the C++ specification.

Recommendation: **Do nothing**

§ 7.2. Excess precision

Excess precision is currently handled in C and C++ via the rules embodied by the setting of the `FLT_EVAL_METHOD` macro (there is no standard way for a user to modify the setting in other pragmas, even with C's pragmas), although there are currently some unclear issues with the current rules, e.g., CWG2752.

Given that compilers do not reliably implement the behavior required by them for `FLT_EVAL_METHOD` on the one platform where it makes a difference (namely, arithmetic using only the x87 FPU on x86 hardware), that this platform is of declining importance, such compilers are today nonconforming and are unlikely to become conforming in response to future standard changes. It is not a worthwhile use of this committee's time to further clarify the rules here if no one is going to change to become conforming.

Recommendation: **Strip out support for excess precision entirely**

§ 7.3. Denormal flushing

The main difficulty with denormal flushing is that because the hardware environment can be affected by link-time flags, it is largely unknowable in too many cases by the compiler whether or not denormal flushing will actually be in effect or not. Based on current hardware trends, the performance benefits of enabling denormal flushing are likely to be nullified in the future. Thus, it is reasonable to assume that, in several years' time, denormal flushing may end up having little practical modern relevance, as has happened with excess precision.

For current compilers to be conforming, denormal flushing can neither be prohibited nor required; additionally, the explicit lack of requirement that compile-time floating-point semantics exactly make runtime semantics serves to make the behavior compliant on architectures where the default runtime environment is changed by link-time flags (and thus intrinsically unknowable at compile-time). Absent some possible clarification on the behavior of `std::numeric_limits::denorm_min()`, there does not seem to be a need to change anything with respect to denormal flushing at this time.

Recommendation: **Do nothing**

§ 7.4. Fast-math

The trouble with representing fast-math semantics is that it is an inherently open class of flags (and compilers will include more than whatever the standard requires) which can be independently toggled. The only language features we have that easily accomodate such capabilities are pragmas or block attributes. However, these approaches do not work well with generic code, as discussed in a previous section.

Some fast-math flags are describable as changing the set of allowable values for a type. For example, the effect of `-ffinite-math-only` is to make NaN and infinity values into trap representations of floating-point numbers. Since they have value effect, they actually map quite nicely to being described with a type wrapper like `finite<T>`, where any operation that would result in a NaN or infinity value (such as `sqrt(-1)`) would instead cause undefined behavior (and to have the desired effect on the optimizer, it is necessary that it cause undefined behavior and have unpredictable results, as opposed to relying on erroneous behavior or other more constrained forms of behavior).

Type wrappers work poorly if there are many flags to be applied. Fortunately, there is not a large number of value-based fast math properties: there are four main classes of special values (`-0`, infinities, quiet NaNs, and signaling NaNs), and even then, many of those combinations do not have great practical value (it is not advantageous to support signaling NaNs but not quiet NaNs, for example). Despite this low number, adding more than one type wrapper, or maybe two if they are not orthogonal, seems inadvisable.

The non-value-based fast-math flags, such as allowing reassociation, do not seem particularly amenable to type wrappers, as their effect is largely in relation to combinations of operations and do not have any clear value-based semantics. In addition, since their effect is to enable certain rewrites of the code, for the most part it is more beneficial that these take effect rather globally, as allowing it to happen for a specific, narrow region of code could instead be effected by just rewriting the code to the desired form. Instead, most uses are more likely to be disable these optimizations for particular sensitive regions rather than to enable them. However, there are two main exceptions, which are covered in their own, subsequent sections.

Recommendations: **Let fast-math flags be conforming compiler extensions, enabled or disabled by command-line flags or existing pragmas not described by the standard. Pursue an approach to make pragmas work better with generic code. Consider adopting a `finite<T>`-like class that makes infinities and NaNs undefined behaviors for instances of that class.**

§ 7.5. Associativity

The main use case for allowing free reassociation of variables is a loop, or other reducing context, that is accumulating one or more results over multiple iterations which could be replaced with some form of parallelized loop body, which necessarily executes the reducing steps in a different order than the regular iteration order implied by a serial execution.

Already in C++, `GENERALIZED_SUM` has sufficient specification to imply reassociation. Where a loop does but one reduction, it can be to use `std::accumulate` or `std::reduce` which by using `GENERALIZED_SUM` already imply reassociation. Similarly, the `std::execution::unseq` execution policy also implies the ability to write a more generic loop that the compiler may vectorize regardless of other legality restrictions. So C++ already has something akin to a free function that will do a reassociable reduction.

Recommendation: **Add no new facilities**

§ 7.6. FMA contraction

Being able to contract expressions into FMAs is arguably the most useful of fast-math flags, since if hardware was FMA instructions, they are almost always better to use than regular instructions. There are, however, cases where FMA contraction is undesirable, so users need to have the ability to opt out of FMA contraction at times.

From a semantic perspective, the best approach to FMA contraction is to provide a distinct free function that is an FMA operation if the hardware can do it quickly or else a pair of multiply/add instructions. For example, the Julia language provides such an operation. The advantage of such an approach is that it is always clear what the user intends. The main disadvantage, though, is that users would have to opt into the new code form, and it also requires more user overloads to make a "floating-point-like" type. Additionally, assuming such a facility is limited to floating-point types, it makes it harder to write basic numerical utilities that are agnostic over their underlying types being integers or floating-point types (or other kinds of algebraic categories).

The primary way this feature is effected today within compilers is via pragmas and command-line options. Like all pragma-based approaches, this suffers from the current inability to write a generic function that can inherit the pragma state of its caller. While IEEE 754 requires for its rules on contraction that it can happen only within an expression, most compiler implementations ignore this rule and instead freely contract any multiply and add that may happen to wander near each other after optimizations kick in, even should they cross function boundaries. Between this and the multiple phases of optimization, the compiler will decide to contract or fail to contract an operation differently even for the same operation in different contexts, and there is a steady trickle of user complaints about this difference that are simply not fixable with this design.

The final major set of alternatives is to make contractability a part of the floating-point type. But contractability is fairly orthogonal to all of the other concerns of floating-point semantics, and as a result, the problem of composing multiple type properties is particularly salient. Furthermore, the feature is a property largely of the operations (and in particular, really just addition and multiplication) and not of the types, which makes expressing it via types a somewhat circuitous way to achieve the goals.

None of these options can be advocated for as particularly good solutions to the problem. Instead, a choice must be made as to which one is the least bad solution.

Recommendation: **Pursue a free function for fast FMA to enable FMA contraction.**

§ 7.7. Constant expression rules

[expr.const/p23](#) lists as recommended practice that floating-point results be consistent between compile-time and runtime execution. Additionally, [library.c/3](#) requires that math functions conform to the requirements of C's Annex F as far as possible for the purposes of constant expressions. These are the only guidelines for floating-point constant expressions in C++ at present.

The generalized constant expression support in C++ allows us to theoretically support accessing and modifying the environment in constant-evaluated contexts. However, given that it is not really possible to synchronize the compile-time and runtime environments, and given that admitting advanced environment features are more difficult for the compiler to emulate correctly at compile-time, it seems most prudent to simply not allow general access to the environment at compile-time. Rounding mode could be supported, but a static rounding mode support as envisioned in P2746 is a superior interface, and there is no need for a general environment access for that feature. Instead, the environment should be fixed to its default for compile-time access.

There is a related question about the behavior of floating-point expressions in the presence of exceptions. C++ already requires that a call to C standard library function that raises a floating-point exception is a non-constant library call; it is possible to extend this rule to apply to all floating-point expressions, even basic ones like addition or multiplication. Some compilers already do this when the result of an operation is a NaN, but this does not appear to happen in the case of overflows or underflows.

Deviations between compile-time and runtime execution can happen for a few reasons. Environment access might be different. The subtle differences in rules around excess precision and denormal flushing can also produce a difference. Finally, the frontend may fail to account for the rewritten code caused by generic fast-math optimizations, especially ones like FMA contraction, as the frontend is not capable of perfectly predicting what the optimizer will do. As a result, it is not really possible to mandate that the compile-time and runtime execution follow the same rules, and likely implementations would simply ignore such a mandate even if it were to exist, due to the intrinsic difficulties in doing so.

Recommendations: **Continue to not require equivalent semantics for compile-time and runtime execution of floating-point. Do not make any floating-point environment manipulation or introspection functions `constexpr`. Explore making floating-point exceptions in regular operations non-`constexpr`.**

§ 7.8. Type traits

C++ has a set of type traits centered around `std::numeric_limits` that indicate the properties of floating-point hardware. All of the members of `std::numeric_limits` are `constexpr`, which mean the compiler has to fix a choice for the value for the entire execution, and the traits are not capable of reflecting the dynamic environment if the hardware is capable of modifying behavior dynamically (e.g., flushing denormals).

With the existing wording, it is not clear what the value of (e.g.) `has_infinity` should be if compiled with fast-math flags that make infinities equivalent to undefined behavior: the format supports it, but the execution environment does not. Given the ability in most implementations to vary the behavior of fast-math-like flags on a finer grained unit than the entire translation unit, the effects of these flags should be similar to the effect of the dynamic floating-point environment on the flags, which is to say none. This is already how implementations interpret these flags, so it should be made clearer in the specification that this is intended behavior.

Recommendation: **Clarify in wording that type traits do not reflect fast-math flags.**

§ 7.9. Rounding mode

Currently, C++ borrows the interface for rounding modes from C, but doesn't adopt the `FENV_ROUND` pragma that was recently added in C23. The issues with the dynamic rounding mode functions are fairly well laid out in the existing series of papers on P2746, which proposes to deprecate the existing functionality and replace it with what are effectively free functions for doing operations in a fixed static rounding mode (including `constexpr` support).

As P2746 has already made substantial progress within WG21, there is no reason to disturb that progress, and it is only mentioned here for the sake of completeness on the topic.

Recommendation: **Continue work on P2746**

§ 7.10. Reproducible results

P3375 proposes that C++ add some feature to indicate reproducible results for the compiler. It is still at an early stage of discussion, and does not have a specific design for the feature, but strongly leans toward a type wrapper or new fundamental type approach.

For the narrow use case of ensuring that the numerical results are identical across diverse compilation environments, attaching this information via a type wrapper or fundamental type works well. The big problem with such approaches is that type properties do not compose well, but a type annotation of "disregard all other instructions to loosen semantics" implies that there is no composition at all--applying fast-math flags to such a type defeats the purpose of the type in the first place. Furthermore, type-based properties work the best at ensuring that they will be picked up in generic code, which is especially important for this use case.

Recommendation: **Pursue a type-based approach that enforces a precise floating-point model for some operations, without the ability to mix with other fast-math flags.**

§ 7.11. Environment access

C++ relies on the C library functions to manage access to the environment, and other than a comment not requiring support for C's `FENV_ACCESS` pragma (necessary for these functions to have effect in C), is silent on any details. In practice, code needing environment access tends to rely on the use of compiler flags to put the compiler in a strict floating-point model.

But even without these flags, it is often possible to get the compiler to mostly reliably support environment access by following two rules. First, ensure that the floating-point code in question is actually executed at runtime rather than being potentially executed at compile-time (which largely means preventing the compile-time optimization of constant-folding from kicking in). Second, ensure that the compiler does not have the freedom (or at least the desire) to move the floating-point code around the calls to the environment function. After all, in the absence of fast-math flags, there tends to be rather little in the way of optimizations that can be applied to floating-point code outside of the usual universal optimizations of constant folding, dead code elimination, and some forms of code motion.

Given these constraints, for the purposes of checking the exceptions of a floating-point operation, the approach that is most worth pursuing is probably something in the form of a library function that wraps a lambda and returns the floating-point exceptions:

```
float scale(float value, float pointScaleFactor) {
    float result;
    // Returns the exceptions, if any, raised by any floating-point operation
    // invoked by the lambda.
```

```

auto exception = std::check_fp_except([&]() {
    result = value * pointScaleFactor;
});
if (exception & (FE_OVERFLOW | FE_UNDERFLOW)) {
    // report error...
}
return result;
}

```

By wrapping the code in what the optimizer will see as a separate function call, there is a natural optimization barrier between the code being checked for exceptions and the code where the exceptions don't matter. The function call can also do the task of clearing any prior exceptions that may have been caused (which is usually necessary anyways). Ideally, the lambda would be specifically compiled in a strict floating-point mode, and this effect would trickle down to all of the functions recursively called by the lambda body, but even if the compiler fails to do this, the natural optimization barrier of the function call will often be sufficient to keep the code working in many production codebases.

An alternative to the library function would be to more directly use a `try/catch` syntax for floating-point exceptions. Mapping that to C++'s existing keywords would create confusion with the existing exception-handling mechanism, as floating-point exceptions work very differently from C++'s exceptions. TS 18661-5 adds a similar `try/catch`-like mechanism for doing alternation floating-point exception handling, but expressed via a cumbersome pragma syntax that has no known implementations and received little support from the broader WG14 committee when portions of the floating-point TSes were integrated in C23. C++ having lambdas enables the library function to replace language keywords or pragmas to enable this feature, although it does not provide a complete solution for enforcing the floating-point model within the lambda.

Using a type-based approach to enable or disable the ability of floating-point code to interact with the environment is an extremely poor match. The environment is inherently a shared (thread-local) data structure, and hardware instructions that touch the environment are not particularly fast. Repeatedly turning on and off for individual operations is not a good idea, outside of rounding mode where static rounding mode instructions exist on many hardware classes (and static rounding mode being integrated into major languages like C++ would push more hardware vendors to include support for it for performance reasons).

Outside of the accessing the currently raised floating-point exceptions, the other major components of the floating-point environment are the current dynamic rounding mode, the denormal flushing mode(s), bits to enable traps on floating-point exceptions, and other bits generally unaccounted for in compilers' model of floating-point. Rounding mode is already discussed earlier, with a preference for relying on static rounding mode rather than a dynamic rounding mode. The trap bits can be viewed as a way to make the overhead of testing for floating-point exceptions cheaper on hardware that supports it, and probably suffice to be used via existing mechanisms. Similarly, the rest of the floating-point environment tends to be poorly modeled by compilers, and to use them effectively, a user already needs to force the compiler into a very strict floating-point model globally. Given that there is little commonality across architectures on these extra bits, code that truly cares about them already needs to rely on implementation-specific features like inline assembly to access them, and the benefit of the language standardizing means of access to them is little.

Recommendation: Pursue a library function that test for floating-point exceptions occurring within the execution of a lambda argument.

§ 8. Questions

- Is WG21 interested in pursuing something like C's Annex F for C++?
- Is WG21 in favor of removing support for excess precision in C++?
- Is WG21 in favor of adding support for methods to dynamically control denormal flushing?
- Is WG21 interested in ways to improve pragma compatibility with generic code?
- Is WG21 interested in a feature like `finite<double>` for representing floating-point types that have UB for non-finite values?
- Is WG21 interested in favor of a `fma_fast`-like function for enabling FMA contraction?
- Is WG21 interested in enabling floating-point environment support in `constexpr` contexts?

- Is WG21 interested in making floating-point exceptions (other than `FE_INEXACT`) fail to be evaluated at compile time?
- Is WG21 interested in adding a function for testing for floating-point exceptions that occur within the evaluation of a lambda argument?